| | | | |
|---|---|---|---|
| **Name:** | **Listana, Lance Ivan R.** | **Student #:** | **202221081** |
| **Year Level & Section:** | **BSIT 3B** | **Date:** | **05-06-25** |

## Module 6: Asynchronous Task

**Title:** Analyzing and Fixing Common Security Flaws in Software Design

**Objective:**
Students will identify and analyze common security flaws related to:

- Design Choices and Vulnerabilities
- Poor Authentication Mechanisms
- Inadequate Access Controls
- Lack of Encryption
- Insecure Network Design
- Error Handling and Input Validation
- Software Architecture Weaknesses
- Logging and Monitoring Deficiencies
- Secure Development Practices

**Part 1:** Vulnerability Exploration

**Step 1:** Set up the OWASP Juice Shop

- Follow the instructions from https://owasp.org/www-project-juice-shop/ to install the application locally or use an online instance (Heroku or Gitpod).

**Step 2:** Identify Security Flaws

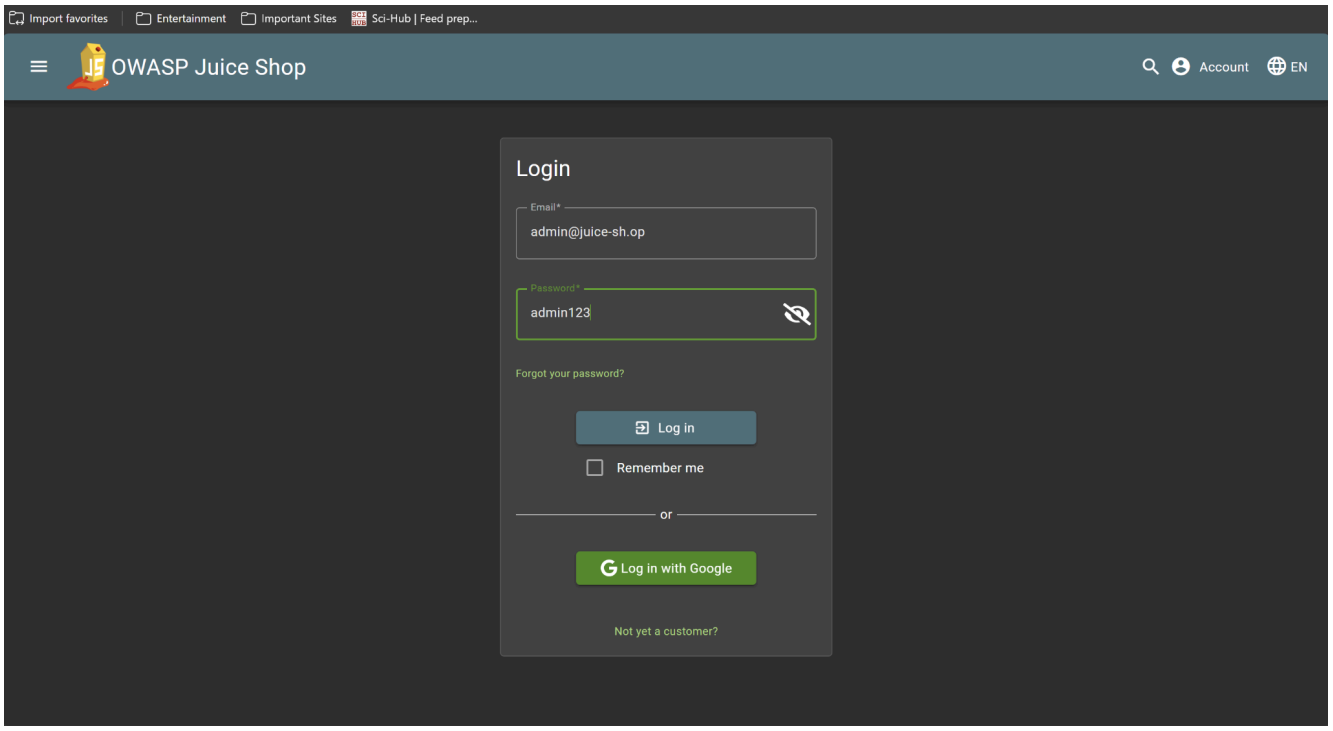
- Use the app and identify examples of the following design flaws. List at least one example for each:

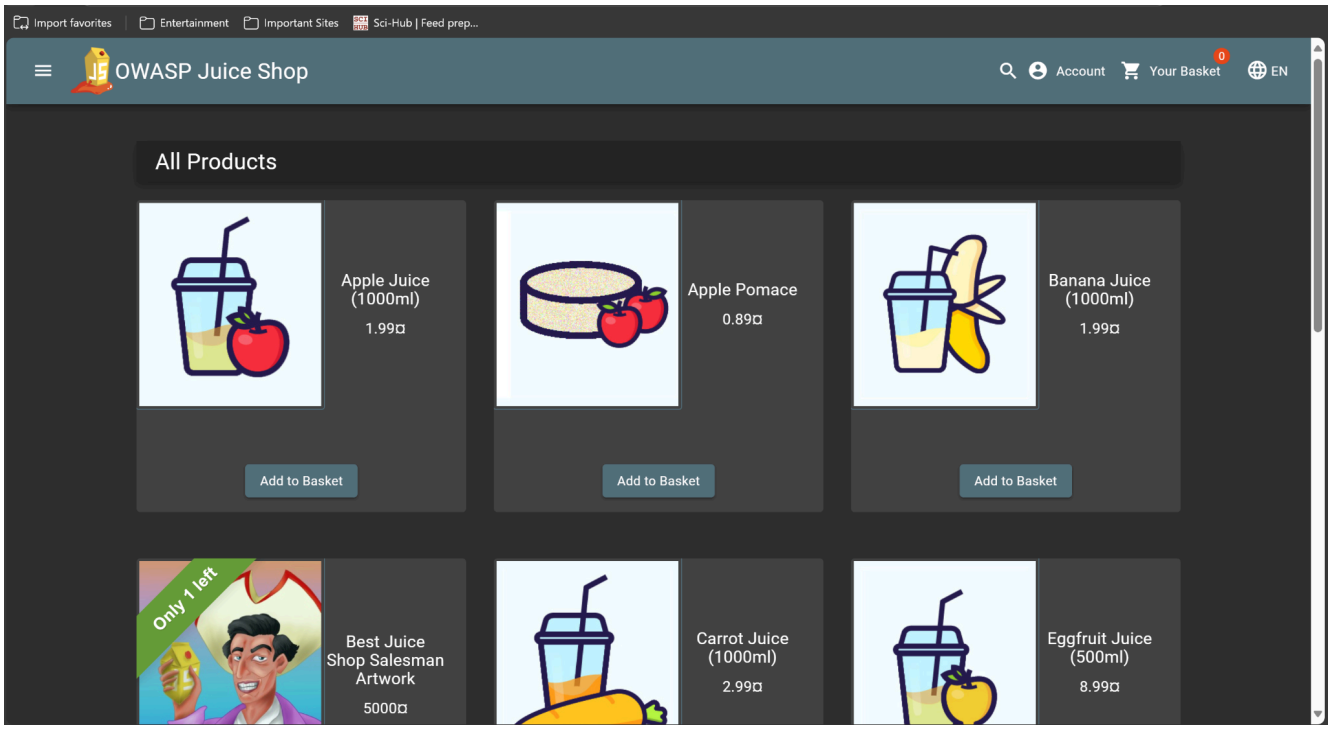
  - Poorly Designed Authentication Mechanism
  - Inadequate Access Control
  - Unencrypted Sensitive Data
  - Network-related vulnerabilities
  - Inadequate Input Validation
  - Error messages leaking technical details
  - Weak architectural decisions (no API gateway or microservice isolation) ▪ Lack of logging/monitoring mechanisms
  - Absence of secure development practices (hardcoded secrets, missing tests)

Answers:

Poorly Designed Authentication Mechanism :

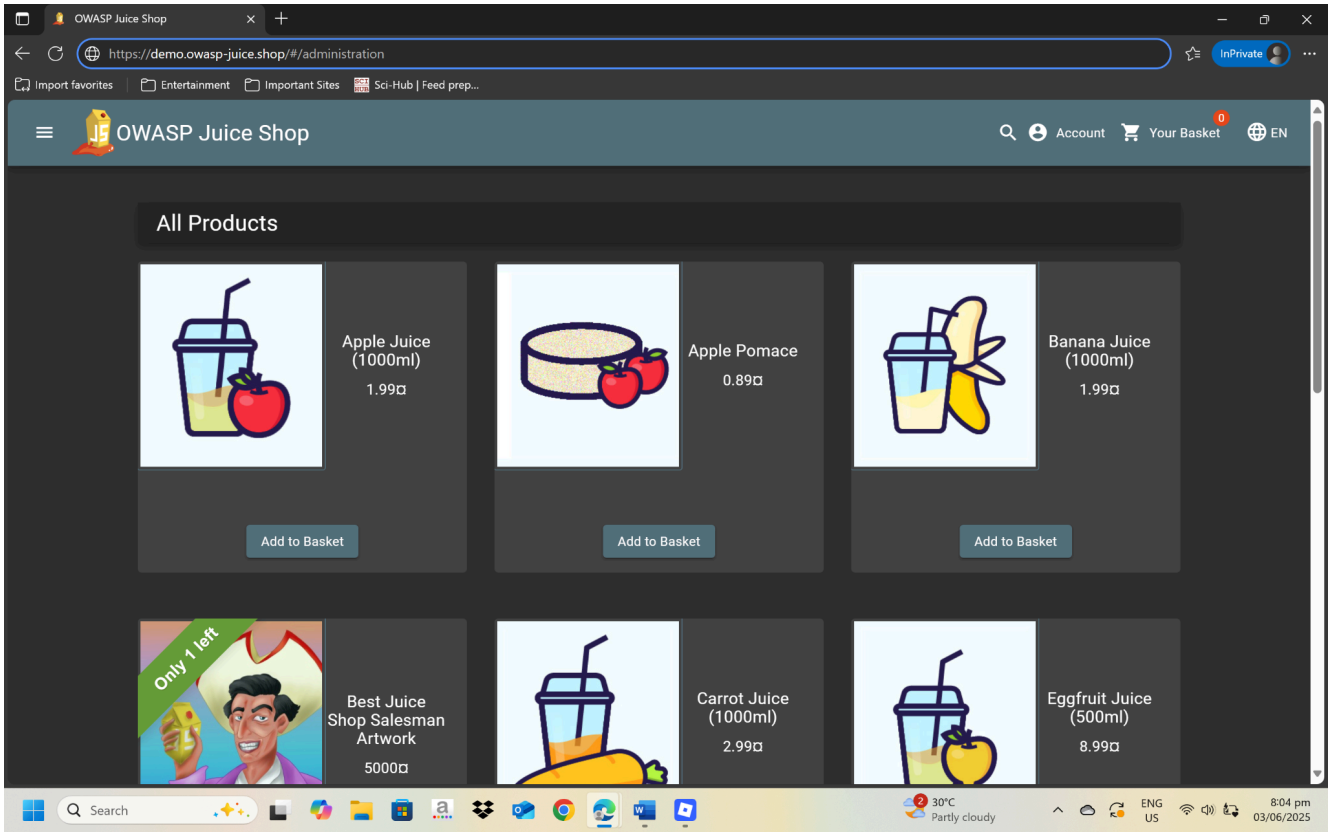
- Logging in using default credentials highlights a flawed authentication design, as it suggests the presence of weak or commonly known default usernames and passwords. This allows attackers to gain unauthorized access without needing to exploit any complex vulnerability, exposing the system to easy compromise.

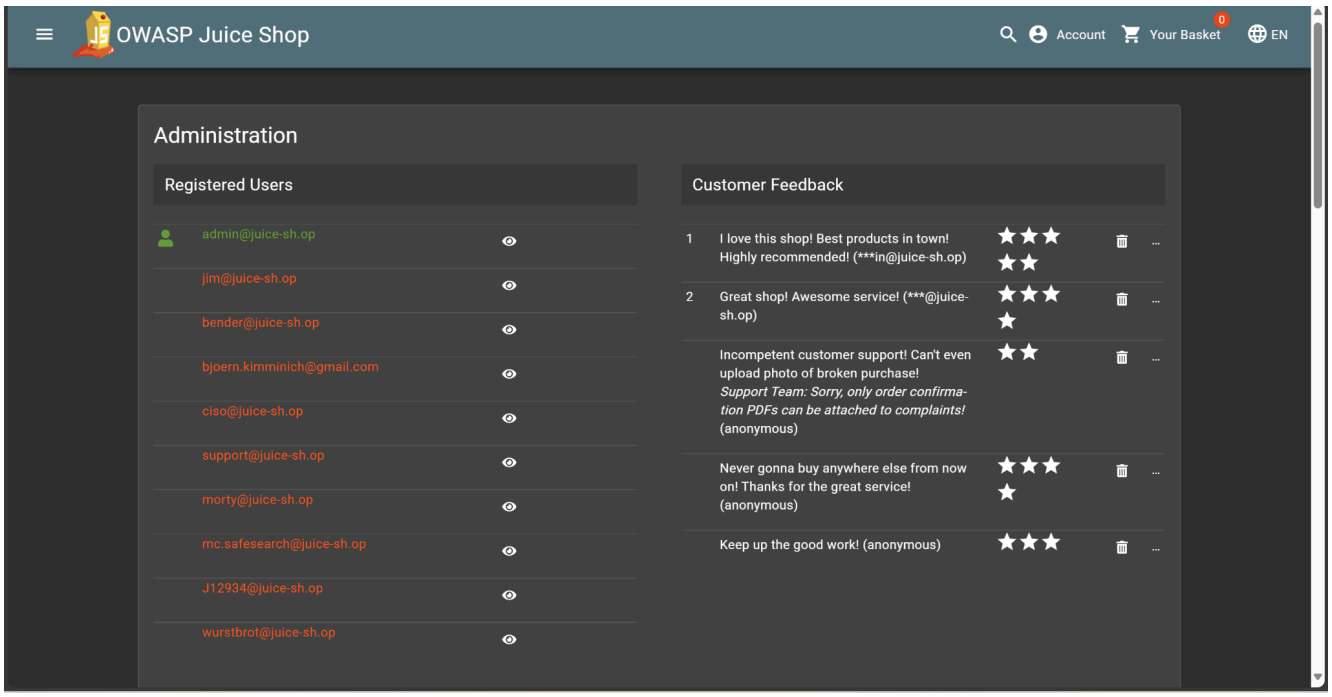


Inadequate Access Control :

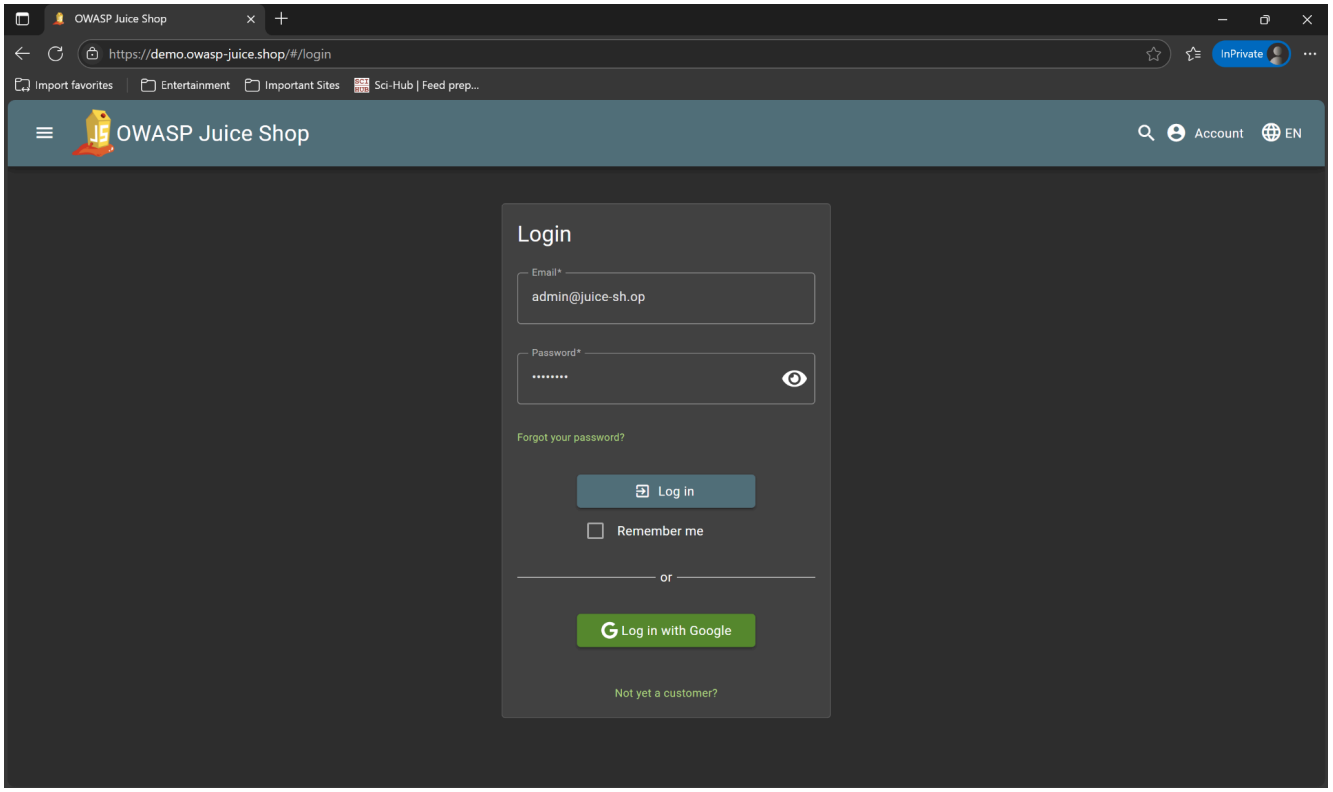
- I can access the admin without proper authorization.

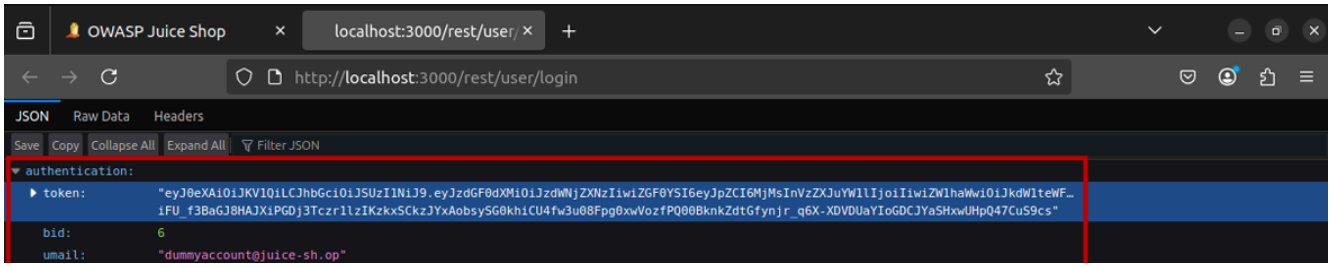



Unencrypted Sensitive Data :

- Discovering a .json file in Developer Tools reveals plain-text credentials such as tokens and emails, indicating that sensitive data is improperly exposed and not securely stored or encrypted as it should be.



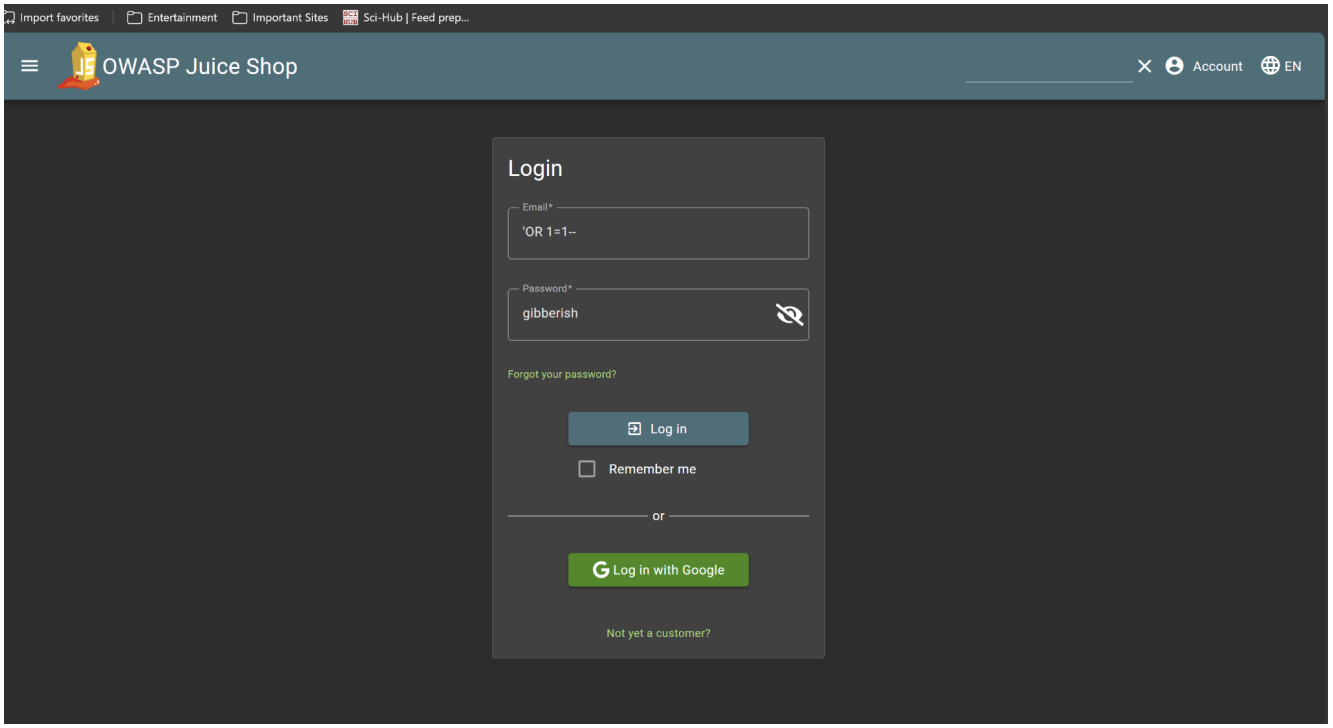




Network-related vulnerabilities :

- HTTP instead of HTTPS and no warning icon or no icon at all at the left side of the address bar – this shows insecure webpage.

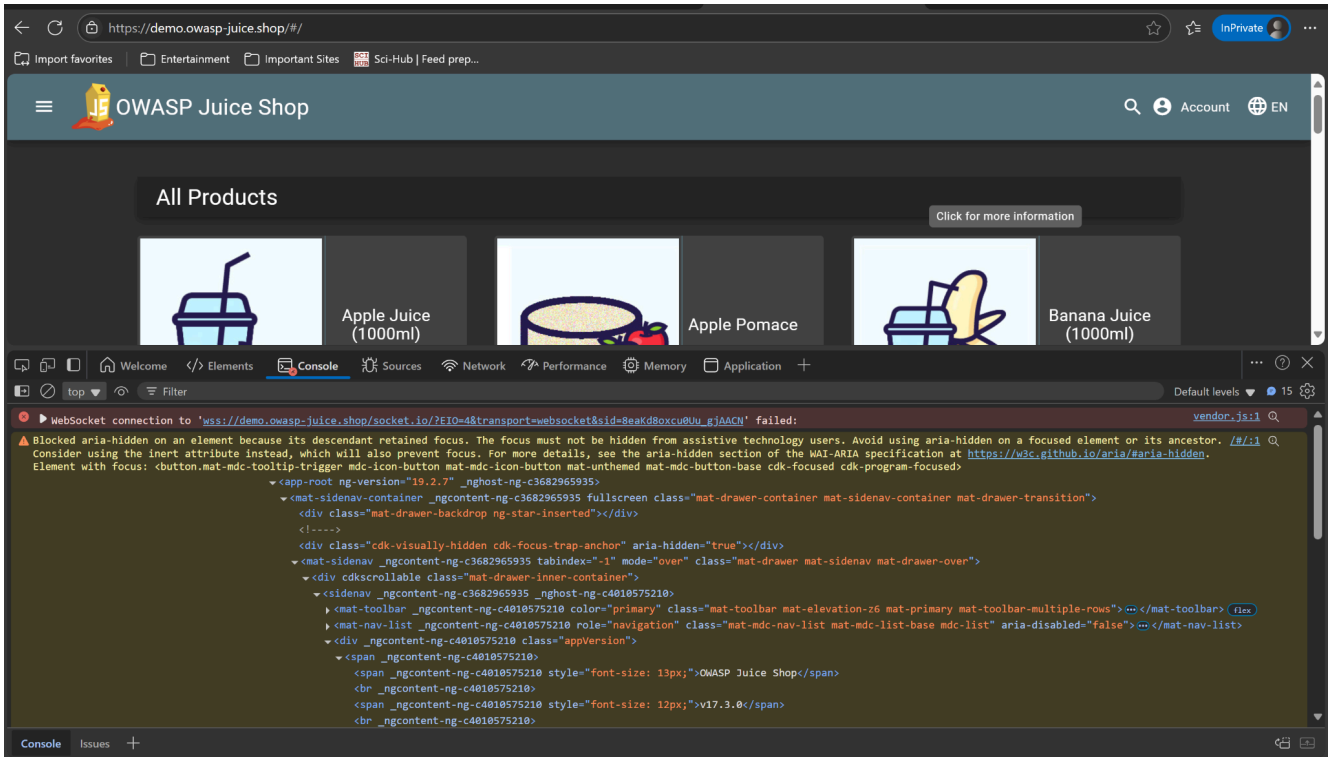
Inadequate Input Validation :

- Using SQL Injection to log in demonstrates that backend SQL queries are not properly secured with parameterization, leaving the application vulnerable to direct manipulation of the database.




Error messages leaking technical details:

- Error messages expose sensitive backend information like file paths, database configurations, or code structure. This unintended disclosure can aid attackers in developing more precise and effective attacks.



**Part 2:** Design and Code Fixes

Choose three issues from the list above and:

1. Describe why the current design is insecure
2. Propose a secure design alternative
3. If applicable, show code-level or architectural improvements

Example:

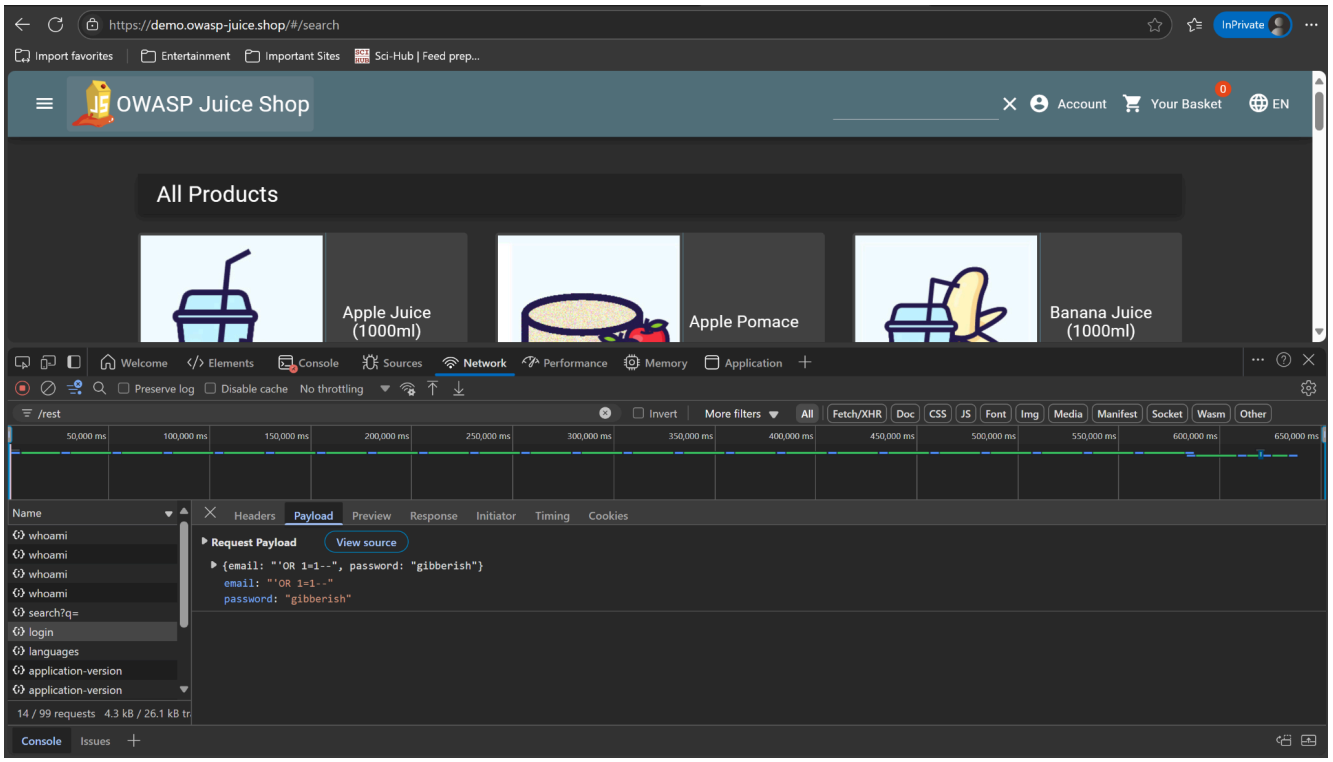
Issue: Passwords are stored in plaintext.

Fix: Store passwords hashed with bcrypt.

Code: hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())

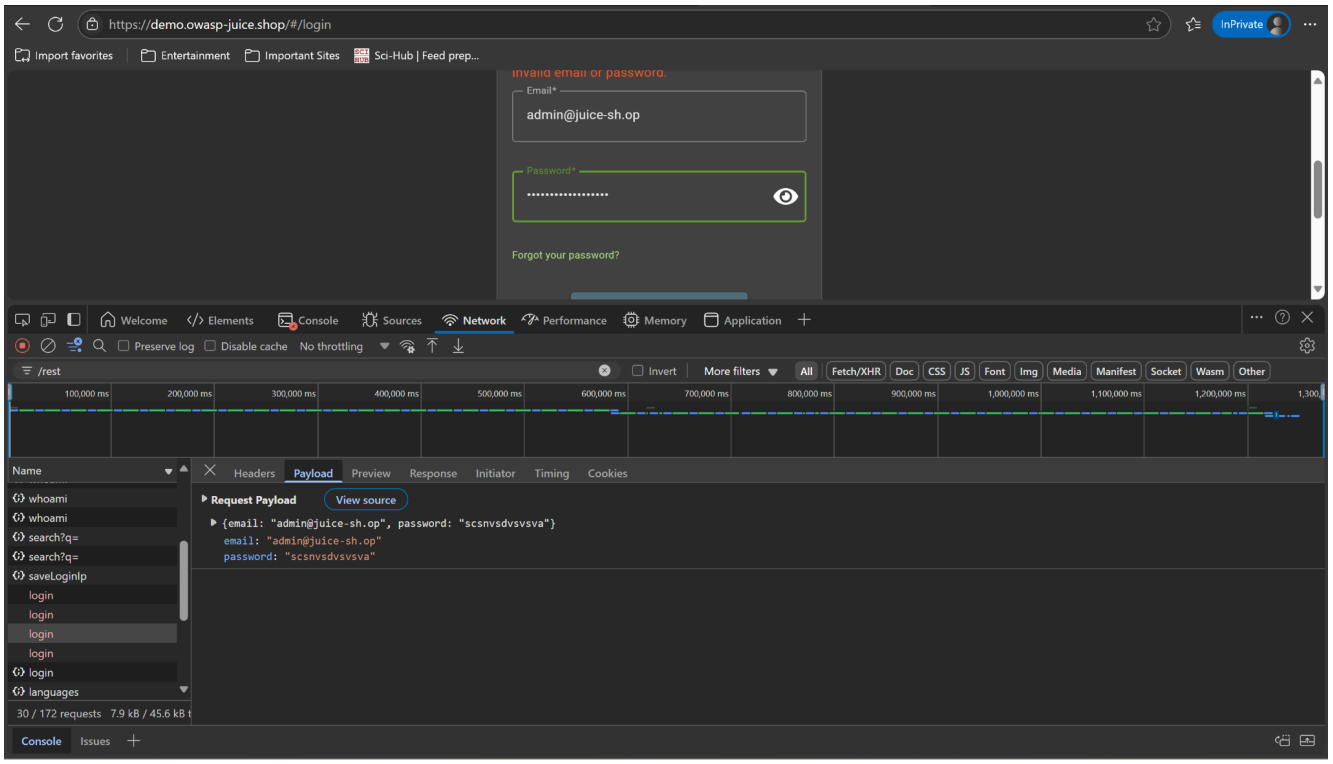
Weak Architectural Decisions (No API Gateway or Microservice Isolation) :

- The frontend connects directly with backend services without an API gateway or service isolation, making the system more vulnerable. Without centralized control mechanisms like rate limiting, authentication, or logging, it's easier for attackers to target and exploit backend endpoints.



Lack of logging/monitoring mechanisms :

- You have unlimited attempts to login even with an incorrect password. And the attempts are visible.



Absence of Secure Development Practices (hardcoded secrets, missing tests):

**ISSUE 1**: Lack of logging/monitoring mechanisms

**Describe why the current design is insecure**

- Juice shop web allows the user to attempt unlimited times.

**Propose a secure design alternative**

- Implement Account Lockout or Temporary Blocking
- Add CAPTCHA After Repeated Failures
- Use Rate Limiting (API Level)

**Code-level or architectural improvements**

```
const express = require("express");

const app = express();

app.use(express.json());


const attempts = {};


app.post("/login", (req, res) => {

  const { username, password } = req.body;

  const now = Date.now();

  const user = attempts[username] || { tries: 0, blockUntil: 0 };


  if (now < user.blockUntil)

    return res.status(429).send("Blocked. Try later.");


  const success = username === "admin" && password === "secure123";


  if (success) attempts[username] = { tries: 0, blockUntil: 0 };

  else {

    user.tries++;

    if (user.tries >= 5) user.blockUntil = now + 15 * 60 * 1000;
```

```
    attempts[username] = user;
  }

  res.send(success ? "Login OK" : "Login failed");
});

app.listen(3000);
```

**Issue 2: Poorly Designed Authentication Mechanism**

- **Describe why the current design is insecure**

  Easy for attackers to gain unauthorized access.

  **Propose a secure design alternative**

  - Lack of Multi-Factor Authentication:
  - Enforce password complexity

**Code-level or architectural improvements :**

```
const passwordSchema = new passwordValidator();
passwordSchema
.is().min(8)
.has().uppercase()
.has().lowercase()
.has().digits()
.has().symbols();
if (!passwordSchema.validate(req.body.password)) {
return res.status(400).json({ error: 'Weak password' });
}
```

**Issue 3:** Inadequate Input Validation

**Describe why the current design is insecure**

Inputs from users are not properly validated or sanitized

**Propose a secure design alternative**

- Sanitize All Inputs
- Reject Unexpected or Extra Fields
- Validate at All Layers

**Code-level or architectural improvements :**

```
const { body } = require('express-validator');


app.post('/feedback', [

  body('comment').trim().escape(),

  body('rating').isInt({ min: 1, max: 5 })

], (req, res) => {

  // sanitized req.body.comment is safe to use

});
```

**Part 3:** Reflection and Submission

Answer the following reflection questions in a document:

1. Which types of flaws did you find most concerning and why?

- Poorly Designed Authentication Mechanism, because it directly enables unauthorized access to user or admin account

2. What secure design principles could have prevented these flaws?

- To prevent weak authentication, it's important to follow secure design principles. For example, locking accounts after too many failed attempts (fail-safe defaults) can stop brute-force attacks. Adding layers like strong

passwords and MFA (defense in depth), and limiting user access to only what they need (least privilege), makes it much harder for attackers to do damage.

3. How can development teams integrate secure practices into their workflow?

- Development teams can build secure practices into their workflow by making security part of every stage, not just the end. For example, using code reviews, automated security testing, and tools like linters or dependency checkers helps catch issues early. Regular training and involving security experts during planning also keep everyone focused on building safer software from the start.

**The document MUST contain the following:**

1. List of identified issues with screenshots
2. Proposed fixes (including code/architecture suggestions)
3. Answers to reflection questions

**Grading Rubric (out of 100 points):**
**Criteria Points** Identified vulnerabilities (9 items, 5 pts each) 45
Proposed secure solutions (3 items, 10 pts each) 30 Reflection and critical thinking 15 Clarity, organization, and presentation 10